LABORATOR 2 – statistica

EX#1 Creaţi un fişier în Python® prin care:

(a) să se simuleze aruncarea cu o monedă corectă i.e. $\mathbb{P}(H) = 1/2, \mathbb{P}(T) = 1/2$;

(b) să se simuleze aruncarea cu o monedă măsluită: $\mathbb{P}(H) = 0.7, \mathbb{P}(T) = 0.3$;

(c) să se simuleze câte N aruncări cu moneda corectă, respectiv măsluită, şi să se estimeze numeric (frecvenţionist), pentru fiecare caz, probabilitatea să cadă H;

(d) să se afişeze într-un sistem xOy graficul estimărilor obţinute la (c) în funcţie de numărul de aruncări $i = \overline{1, N}$.

```
#ex1
import numpy as np
import matplotlib.pyplot as plt
#b+c
N = int(1e2)
x = np.random.rand(N) #Generează N numere aleatorii uniform în [0,1]
→ simulăm aruncări cu moneda corectă
H = 1*(x<0.7)
P = np.cumsum(H)/np.arange(1,N+1)
#np.cumsum(H) = suma cumulativă a lui H până la fiecare aruncare

#Ex.: [1, 1, 2, 3, ...]

#np.arange(1, N+1) = [1, 2, 3, ..., N] → numărul total de aruncări
efectuate până în acel moment

#Împărţind: P[i] = (număr total H până la aruncarea i) / (număr total
aruncări i)
```

```
plt.figure()
plt.plot(P,'.')
plt.grid()

N = 10000  # numărul de aruncări
p_fair = 0.5   # probabilitatea de H pentru moneda corectă
p_biased = 0.7 # probabilitatea de H pentru moneda măsluită
#a
x_fair = np.random.rand(N)
H_fair = 1 * (x_fair < p_fair)
print(np.sum(H_fair)/N)
#b
x_masluit = np.random.rand(N)
H_masluit = 1*(x_masluit < 0.7)
print(np.sum(H_masluit)/N)
```

EX#2 Creaţi un fişier în Python® prin care:

(a) să se simuleze aruncarea cu un zar corect i.e. $\mathbb{P}(\{i\}) = 1/6, i = \overline{1,6}$;

(b) să se simuleze aruncarea cu un zar măsluit: $\mathbb{P}(\{1\}) = 0.25, \mathbb{P}(\{2\}) = 0.25, \mathbb{P}(\{3\}) = 0.2, \mathbb{P}(\{4\}) = 0.15, \mathbb{P}(\{5\}) = 0.1, \mathbb{P}(\{6\}) = 0.05$;

(c) să se simuleze câte N aruncări cu zarul corect, respectiv măsluit, şi să se estimeze numeric (frecvenţionist), pentru fiecare caz, probabilitatea să se obţină faţa $\{i\}$, pentru $i \in \{1, 2, 3, 4, 5, 6\}$ fixat.

```
--ex2
import numpy as np
import matplotlib.pyplot as plt

# (a) Zar corect
N = int(1e5)
x = np.random.rand(N)

v1 = 1 * (x < 1/6)
v2 = 1 * ((1/6 <= x) & (x < 2/6))
v3 = 1 * ((2/6 <= x) & (x < 3/6))
v4 = 1 * ((3/6 <= x) & (x < 4/6))
v5 = 1 * ((4/6 <= x) & (x < 5/6))
v6 = 1 * ((5/6 <= x) & (x <= 1))

print("ZAR CORECT:")
print(np.sum(v1)/N)
print(np.sum(v2)/N)
print(np.sum(v3)/N)
print(np.sum(v4)/N)
print(np.sum(v5)/N)
print(np.sum(v6)/N)

#b
# (b) Zar măsluit
```

```
N = int(1e2)
x = np.random.rand(N)

v1 = 1 * (x < 0.25)
v2 = 1 * ((0.25 <= x) & (x < 0.50))
v3 = 1 * ((0.50 <= x) & (x < 0.70))
v4 = 1 * ((0.70 <= x) & (x < 0.85))
v5 = 1 * ((0.85 <= x) & (x < 0.95))
v6 = 1 * ((0.95 <= x) & (x <= 1))

print("ZAR MĂSLUIT:")
print(np.sum(v1)/N)
print(np.sum(v2)/N)
print(np.sum(v3)/N)
print(np.sum(v4)/N)
print(np.sum(v5)/N)
print(np.sum(v6)/N)

# (C)
P1 = np.cumsum(v1) / np.arange(1, N+1)
P2 = np.cumsum(v2) / np.arange(1, N+1)
P3 = np.cumsum(v3) / np.arange(1, N+1)
P4 = np.cumsum(v4) / np.arange(1, N+1)
P5 = np.cumsum(v5) / np.arange(1, N+1)
P6 = np.cumsum(v6) / np.arange(1, N+1)
```

```
plt.figure()
plt.plot(P1, '.')
plt.plot(P2,'o')
plt.plot(P3,'-')
plt.plot(P4,'v')
plt.plot(P5,'<')
plt.plot(P6,'>')

plt.title("Estimarea probabilităţii pentru faţa 1 (zar măsluit)")
plt.grid()
plt.show()
```

---var2!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------
# (a) Zar corect
# ------------------------------------
N = int(1e5)
x = np.random.rand(N)
```

```
v1_f = 1 * (x < 1/6)
v2_f = 1 * ((1/6 <= x) & (x < 2/6))
v3_f = 1 * ((2/6 <= x) & (x < 3/6))
v4_f = 1 * ((3/6 <= x) & (x < 4/6))
v5_f = 1 * ((4/6 <= x) & (x < 5/6))
v6_f = 1 * ((5/6 <= x) & (x <= 1))

print("ZAR CORECT:")
print(np.sum(v1_f)/N)
print(np.sum(v2_f)/N)
print(np.sum(v3_f)/N)
print(np.sum(v4_f)/N)
print(np.sum(v5_f)/N)
print(np.sum(v6_f)/N)

# ------------------------------------
# (b) Zar măsluit
# ------------------------------------
x = np.random.rand(N)

v1_b = 1 * (x < 0.25)
v2_b = 1 * ((0.25 <= x) & (x < 0.50))
v3_b = 1 * ((0.50 <= x) & (x < 0.70))
v4_b = 1 * ((0.70 <= x) & (x < 0.85))
v5_b = 1 * ((0.85 <= x) & (x < 0.95))
```

```
v6_b = 1 * ((0.95 <= x) & (x <= 1))

print("\nZAR MĂSLUIT:")
print(np.sum(v1_b)/N)
print(np.sum(v2_b)/N)
print(np.sum(v3_b)/N)
print(np.sum(v4_b)/N)
print(np.sum(v5_b)/N)
print(np.sum(v6_b)/N)

# -------------------------------------
# (c) Estimare probabilităţi pentru zarul măsluit
# -------------------------------------

P1 = np.cumsum(v1_b) / np.arange(1, N+1)
P2 = np.cumsum(v2_b) / np.arange(1, N+1)
P3 = np.cumsum(v3_b) / np.arange(1, N+1)
P4 = np.cumsum(v4_b) / np.arange(1, N+1)
P5 = np.cumsum(v5_b) / np.arange(1, N+1)
P6 = np.cumsum(v6_b) / np.arange(1, N+1)

# -------------------------------------
# (d) Grafic
# -------------------------------------

plt.figure()
```

```
plt.plot(P1, '.', markersize=3, label="Faţa 1")
plt.plot(P2, 'o', markersize=3, label="Faţa 2")
plt.plot(P3, '-', linewidth=1, label="Faţa 3")
plt.plot(P4, 'v', markersize=3, label="Faţa 4")
plt.plot(P5, '<', markersize=3, label="Faţa 5")
plt.plot(P6, '>', markersize=3, label="Faţa 6")

plt.title("Estimarea probabilităţilor pentru fiecare faţă (zar măsluit)")
plt.legend()
plt.grid()
plt.show()
```

pentru $i \in \{1, 2, 3, 4, 5, 6\}$ fixat.

EX#3 Creaţi un fişier în Python® prin care:

(a) să se estimeze numeric (frecvenţionist) probabilitatea ca din 10 de auncări cu o monedă corectă să pice cel puţin o secvenţă consecutivă de (H, H, H, H) sau (T, T, T, T);

(b) să se estimeze numeric (frecvenţionist) probabilitatea ca din 20 de auncări cu o monedă corectă să pice cel puţin o secvenţă consecutivă de (H, H, H, H) sau (T, T, T, T);

(c) să se estimeze numeric (frecvenţionist) probabilitatea ca din 100 de auncări cu o monedă corectă să pice cel puţin o secvenţă consecutivă de (H, H, H, H, H, H, H, H) sau (T, T, T, T, T, T, T, T).

EX#4 Un joc pentru 2 persoane presupune 2 zaruri: unul roşu cu feţele $\{1, 4, 4, 4, 4, 4\}$, unul

!!!!!!!!!!!!!!EX3!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```
import numpy as np

# parametri
nr_aruncari = 10      # 10 aruncări
length = 4      # secvenţă de 4 identice
```

```
Nsim = 200000        # număr de simulări

count = 0
for _ in range(Nsim):
    tosses = np.random.randint(0, 2, size=nr_aruncari)  # 1 = H, 0 = T
    # verificăm toate ferestrele consecutive de lungime run_length
    for i in range(nr_aruncari - length + 1):
        window = tosses[i:i+length]
        if np.all(window == 1) or np.all(window == 0):  # HHHH sau TTTT
            count += 1
            break   # o singură secvenţă este suficientă
p_a = count / Nsim
print("Probabilitate (10 aruncări, secvenţă 4 H sau T):", p_a)


#import numpy as np

# parametri pentru subpunctul (b)
nr_aruncari = 20      # 20 aruncări
length = 4       # secvenţă de 4 identice
Nsim = 200000        # număr de simulări

count = 0
for _ in range(Nsim):
    tosses = np.random.randint(0, 2, size=nr_aruncari)
    for i in range(nr_aruncari - length + 1):
```

```
        window = tosses[i:i+length]
        if np.all(window == 1) or np.all(window == 0):
            count += 1
            break

p_b = count / Nsim
print("Probabilitate (20 aruncări, secvenţă 4 H sau T):", p_b)



# parametri pentru subpunctul (c)
nr_aruncari = 100      # 100 aruncări
length = 8             # secvenţă de 8 identice
Nsim = 200000          # număr de simulări

count = 0
for _ in range(Nsim):
    tosses = np.random.randint(0, 2, size=nr_aruncari)
    for i in range(nr_aruncari - length + 1):
        window = tosses[i:i+length]
        if np.all(window == 1) or np.all(window == 0):
            count += 1
            break

p_c = count / Nsim
print("Probabilitate (100 aruncări, secvenţă 8 H sau T):", p_c)
```

EX#4 Un joc pentru 2 persoane presupune 3 zaruri: unul roşu cu feţele $\{1, 4, 4, 4, 4, 4\}$, unul verde cu feţele $\{3, 3, 3, 3, 3, 6\}$ şi unul negru cu feţele $\{2, 2, 2, 5, 5, 5\}$. Un jucător alege un zar dintre cele 3, iar al doilea alege unul dintre cele 2 zaruri rămase. Ambii jucători aruncă cu zarurile alese şi câştigă cel care obţine numărul mai mare. Creaţi un fişier în Python® prin care:

(a) să se simuleze jocul de mai sus;

(b) să se estimeze numeric (frecvenţionist) probabilitatea ca cel care alege primul să câştige, pentru fiecare alegere posibilă.

-------ex4!!!!!!!!!!!!

```
import numpy as np


# zarurile
zar_rosu   = np.array([1, 4, 4, 4, 4, 4])
zar_verde  = np.array([3, 3, 3, 3, 3, 6])
zar_negru  = np.array([2, 2, 2, 5, 5, 5])



def joc_2_jucatori(zar1, zar2, Nsim=100000):
    """
    Simulează Nsim runde între 2 jucători:
    - zar1 = zar ales de primul jucător
    - zar2 = zar ales de al doilea jucător
    Returnează probabilitatea ca primul jucător să câştige.
    """
    # generăm Nsim aruncări pentru fiecare zar
```

```
    aruncari1 = np.random.choice(zar1, size=Nsim)
    aruncari2 = np.random.choice(zar2, size=Nsim)

    # cine câştigă?
    castiga1 = np.sum(aruncari1 > aruncari2)
    castiga2 = np.sum (aruncari2 > aruncari1)

    # probabilitate frecventistă
    p = castiga1 / Nsim
    p1 = castiga2 / Nsim
    print(p)
    print(p1)
    return p


Nsim = 100000

# 1️⃣ Primul alege roşu
p_rosu_verde = joc_2_jucatori(zar_rosu, zar_verde, Nsim)
p_rosu_negru = joc_2_jucatori(zar_rosu, zar_negru, Nsim)
p_rosu = (p_rosu_verde + p_rosu_negru)/2
print("Probabilitate ca primul să câştige dacă alege roşu:", p_rosu)

# 2️⃣ Primul alege verde
p_verde_rosu = joc_2_jucatori(zar_verde, zar_rosu, Nsim)
p_verde_negru = joc_2_jucatori(zar_verde, zar_negru, Nsim)
```

```
p_verde = (p_verde_rosu + p_verde_negru)/2
print("Probabilitate ca primul să câştige dacă alege verde:", p_verde)

# 3️⃣ Primul alege negru
p_negru_rosu = joc_2_jucatori(zar_negru, zar_rosu, Nsim)
p_negru_verde = joc_2_jucatori(zar_negru, zar_verde, Nsim)
p_negru = (p_negru_rosu + p_negru_verde)/2
print("Probabilitate ca primul să câştige dacă alege negru:", p_negru)

#Dacă al doilea alege roşu → probabilitate ca negru să câştige = p_negru_rosu
#Dacă al doilea alege verde → probabilitate ca negru să câştige = p_negru_verde

#p_negru - reprezintă probabilitatea medie, dacă al doilea jucător alege la întâmplare unul dintre cele două zaruri rămase.
```

--var 2

```
import numpy as np
import matplotlib.pyplot as plt

# Definim zarurile
zar_rosu   = np.array([1, 4, 4, 4, 4, 4])
zar_verde  = np.array([3, 3, 3, 3, 3, 6])
zar_negru  = np.array([2, 2, 2, 5, 5, 5])
```

```
Nsim = 100000  # număr de simulări

def joc_2_jucatori(zar1, zar2, Nsim=Nsim):
    aruncari1 = np.random.choice(zar1, size=Nsim)
    aruncari2 = np.random.choice(zar2, size=Nsim)
    castiga1 = np.sum(aruncari1 > aruncari2)
    return castiga1 / Nsim

# Calculăm probabilitățile pentru fiecare alegere a primului jucător
# Primul alege roșu
p_rosu = (joc_2_jucatori(zar_rosu, zar_verde) +
joc_2_jucatori(zar_rosu, zar_negru))/2
# Primul alege verde
p_verde = (joc_2_jucatori(zar_verde, zar_rosu) +
joc_2_jucatori(zar_verde, zar_negru))/2
# Primul alege negru
p_negru = (joc_2_jucatori(zar_negru, zar_rosu) +
joc_2_jucatori(zar_negru, zar_verde))/2

# Pregătim datele pentru plot
zaruri = ['Rosu', 'Verde', 'Negru']
probabilitati = [p_rosu, p_verde, p_negru]

# Bar plot
plt.figure()
plt.bar(zaruri, probabilitati, color=['red','green','black'])
```

```
plt.ylim(0,1)
#plt.ylabel("Probabilitate câştig primul jucător")
plt.title("Avantajul primului jucător în funcţie de zarul ales")
#plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```

1️⃣ Funcţia plt.bar()

Este folosită pentru a crea un bar plot (grafic cu bare verticale).

Sintaxa principală:

```
plt.bar(x, height, ...)
```

x → poziţiile barelor pe axa x (pot fi etichete sau poziţii numerice)

height → înălţimea fiecărei bare (valorile pe care vrem să le afişăm)

2️⃣ Argumentele din exemplu

zaruri

Este lista cu etichetele pentru bare: ['Rosu', 'Verde', 'Negru']

Acestea apar pe axa x sub fiecare bară

probabilitati

Este lista cu valorile care definesc înălţimea fiecărei bare: [p_rosu, p_verde, p_negru]

Axa y va arăta probabilitatea ca primul jucător să câştige

color=['red','green','black']

Defineşte culoarea fiecărei bare

Prima bară (Rosu) va fi roşie, a doua (Verde) verde, a treia (Negru) neagră